

```
In [4]: # Micro-Project 2 should use visualization libraries, such as, Matplotlib, Seaborn,
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

df = pd.read_csv(
    r'C:\Users\jonat\OneDrive\Desktop\NU Homework\ANA500\ANA500Project\ANA500Project.csv',
    parse_dates=['DateTime'],
    index_col='DateTime'
)

print(df.shape)
df.head(10)

amp_cols = [c for c in df.columns if 'Marrakech' not in c] # Laayoune/Boujdour/Fouknine
kw_cols = [c for c in df.columns if 'Marrakech' in c] # already in kW

# ---- Unit conversion: Amps -> kW ----
# ASSUMPTION: all Ampere-metered zones are
# treated as single-phase 220V connections with an assumed power factor of 0.9,
# since the dataset does not specify voltage, phase, or a domestic/industrial split
# per zone. This is a stated limitation, not a measured fact.
VOLTAGE = 220 # V ac, single-phase
POWER_FACTOR = 0.9 # assumed

def amps_to_kw(current_amps, voltage=VOLTAGE, power_factor=POWER_FACTOR):
    return (voltage * current_amps * power_factor) / 1000

df_kw = df.copy()
df_kw[amp_cols] = df_kw[amp_cols].apply(amps_to_kw)
# kw_cols (Marrakech) are left untouched - already in kW

df_kw.head(10)
```

(29634, 17)

Out[4]:

	zone1_Laayoune	zone2_Laayoune	zone3_Laayoune	zone4_Laayoune	zone5_La
DateTime					
2022-09-14 17:00:00	14.646063	24.066888	31.718142	23.954428	20
2022-09-14 17:30:00	15.678060	24.893525	32.018472	23.612675	20
2022-09-14 18:00:00	16.800154	26.260109	33.291824	24.069451	20
2022-09-14 18:30:00	18.339826	27.586300	34.409672	24.631821	21
2022-09-14 19:00:00	20.861580	31.804195	39.710173	28.069974	26
2022-09-14 19:30:00	21.517977	32.578772	39.914996	28.869993	27
2022-09-14 20:00:00	21.642160	31.592630	39.529048	28.129166	27
2022-09-14 20:30:00	21.047420	31.353166	39.459929	27.918284	27
2022-09-14 21:00:00	21.023086	31.492129	39.096779	27.712305	28
2022-09-14 21:30:00	21.255071	31.192731	38.666592	27.679974	28

In [5]: *#Step Peak vs normal classification (checking what is in the 90th percentile per ci*

```
city_map = {
    'Laayoune': [c for c in df_kw.columns if 'Laayoune' in c],
    'Boujdour': [c for c in df_kw.columns if 'Boujdour' in c],
    'FoumEloued': [c for c in df_kw.columns if 'FoumEloued' in c],
    'Marrakech': [c for c in df_kw.columns if 'Marrakech' in c],
}

# Total Load per city = sum of that city's zone columns (in kW, post-conversion)
for city, cols in city_map.items():
    df_kw[f'total_{city}'] = df_kw[cols].sum(axis=1, skipna=True)

# Peak = Load exceeds the 90th percentile of THAT city's own distribution
labels = pd.DataFrame(index=df_kw.index)
for city in city_map:
    threshold = df_kw[f'total_{city}'].quantile(0.90)
    labels[f'peak_{city}'] = (df_kw[f'total_{city}'] > threshold).map({True: 'Peak'})
    print(f'{city}: 90th percentile threshold = {threshold:.2f} kW')

df_kw = df_kw.join(labels)
df_kw[[c for c in df_kw.columns if 'total_' in c or 'peak_' in c]].head(10)
```

Laayoune: 90th percentile threshold = 145.20 kW

Boujdour: 90th percentile threshold = 37.46 kW

FoumEloued: 90th percentile threshold = 129.85 kW

Marrakech: 90th percentile threshold = 1635.00 kW

Out[5]:

	total_Laayoune	total_Boujdour	total_FoumEloued	total_Marrakech	peak_Laayoune
DateTime					

DateTime	total_Laayoune	total_Boujdour	total_FoumEloued	total_Marrakech	peak_Laayoune
2022-09-14 17:00:00	114.647820	26.803198	168.314692	0.0	Nc
2022-09-14 17:30:00	116.499817	26.879040	168.781371	0.0	Nc
2022-09-14 18:00:00	121.036208	27.850646	171.818346	0.0	Nc
2022-09-14 18:30:00	126.276927	27.491313	173.566312	0.0	Nc
2022-09-14 19:00:00	146.969838	27.003262	171.354033	0.0	
2022-09-14 19:30:00	149.960005	27.774984	173.514641	0.0	
2022-09-14 20:00:00	147.937502	32.322967	175.959425	0.0	
2022-09-14 20:30:00	147.212396	34.746930	177.301145	0.0	
2022-09-14 21:00:00	147.468064	35.391510	172.971298	0.0	
2022-09-14 21:30:00	147.208436	37.175091	172.054252	0.0	

```
In [6]: # Total load over time, colored by peak/normal per city
fig, axes = plt.subplots(len(city_map), 1, figsize=(14, 3 * len(city_map)), sharex=
for ax, city in zip(axes, city_map):
    sns.scatterplot(
        data=df_kw, x=df_kw.index, y=f'total_{city}',
        hue=f'peak_{city}', palette={'Peak': 'crimson', 'Normal': 'steelblue'},
        s=8, alpha=0.5, ax=ax, legend=(ax is axes[0])
    )
    ax.set_title(f'{city} - Total Load (kW) with Peak/Normal Classification')
    ax.set_ylabel('kW')

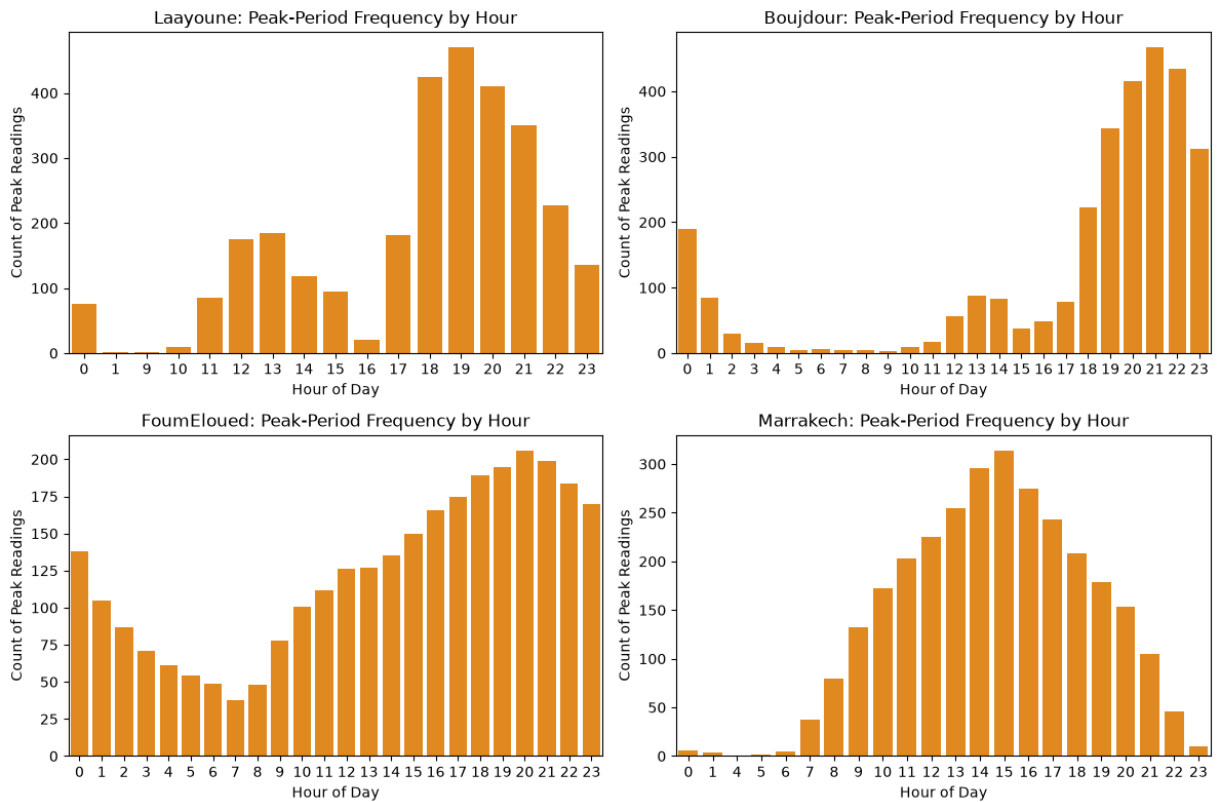
plt.tight_layout()
plt.show()
```



```
In [7]: # Peak hour distribution
df_kw['hour'] = df_kw.index.hour

fig, axes = plt.subplots(2, 2, figsize=(12, 8))
for ax, city in zip(axes.flat, city_map):
    peak_by_hour = df_kw[df_kw[f'peak_{city}'] == 'Peak'].groupby('hour').size()
    sns.barplot(x=peak_by_hour.index, y=peak_by_hour.values, ax=ax, color='darkorange')
    ax.set_title(f'{city}: Peak-Period Frequency by Hour')
    ax.set_xlabel('Hour of Day')
    ax.set_ylabel('Count of Peak Readings')

plt.tight_layout()
plt.show()
```



```
In [8]: # Checking peak and normal distribution based on each zone within a city
zone_cols = [c for c in df_kw.columns if c.startswith('zone')] # all 17 zone columns

zone_peak_labels = pd.DataFrame(index=df_kw.index)
zone_thresholds = {}

for col in zone_cols:
    thresh = df_kw[col].quantile(0.90)
    zone_thresholds[col] = thresh
    zone_peak_labels[f'peak_{col}'] = (df_kw[col] > thresh).map({True: 'Peak', False: 'Normal'})

df_kw = df_kw.join(zone_peak_labels)

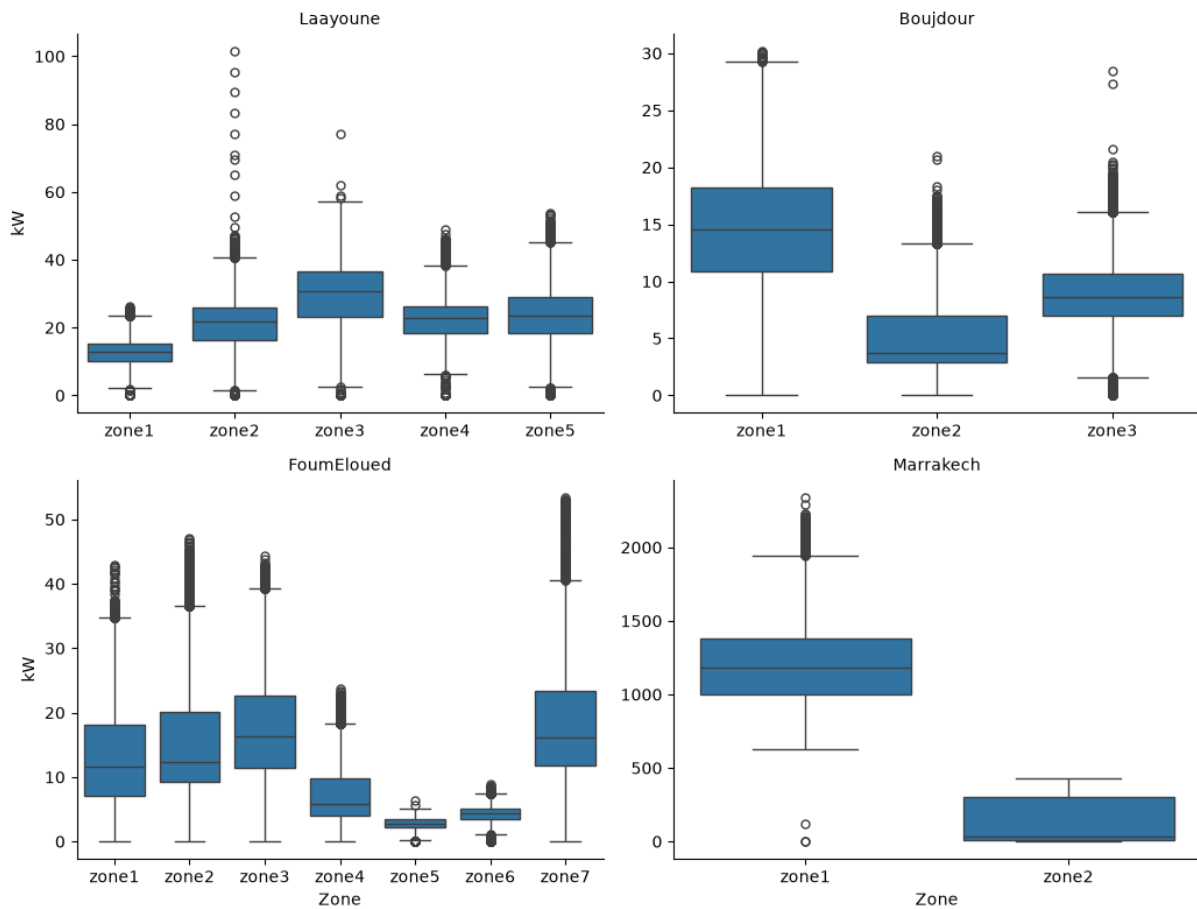
pd.Series(zone_thresholds).sort_values(ascending=False)
```

```
Out[8]: zone1_Marrakech      1587.000000
        zone2_Marrakech      356.000000
        zone3_Laayoune       40.268738
        zone5_Laayoune       32.277793
        zone2_Laayoune       30.302057
        zone7_Foumeloued     30.139766
        zone3_Foumeloued     29.769995
        zone4_Laayoune       28.489016
        zone2_Foumeloued     28.478764
        zone1_Foumeloued     23.660028
        zone1_Boujdour       21.053623
        zone1_Laayoune       17.966850
        zone4_Foumeloued     14.224980
        zone3_Boujdour       12.837948
        zone2_Boujdour       10.008675
        zone6_Foumeloued      5.686602
        zone5_Foumeloued      3.929762
        dtype: float64
```

```
In [13]: # Distribution comparison across all zones (boxplots grouped by city):

# Reshape to Long format: one row per (timestamp, zone) – makes faceted plotting ea
long_df = df_kw[zone_cols].reset_index().melt(
    id_vars='DateTime', var_name='zone_city', value_name='kw'
)
long_df['city'] = long_df['zone_city'].str.extract(r'zone\d+(\w+)')
long_df['zone'] = long_df['zone_city'].str.extract(r'(zone\d+)_')

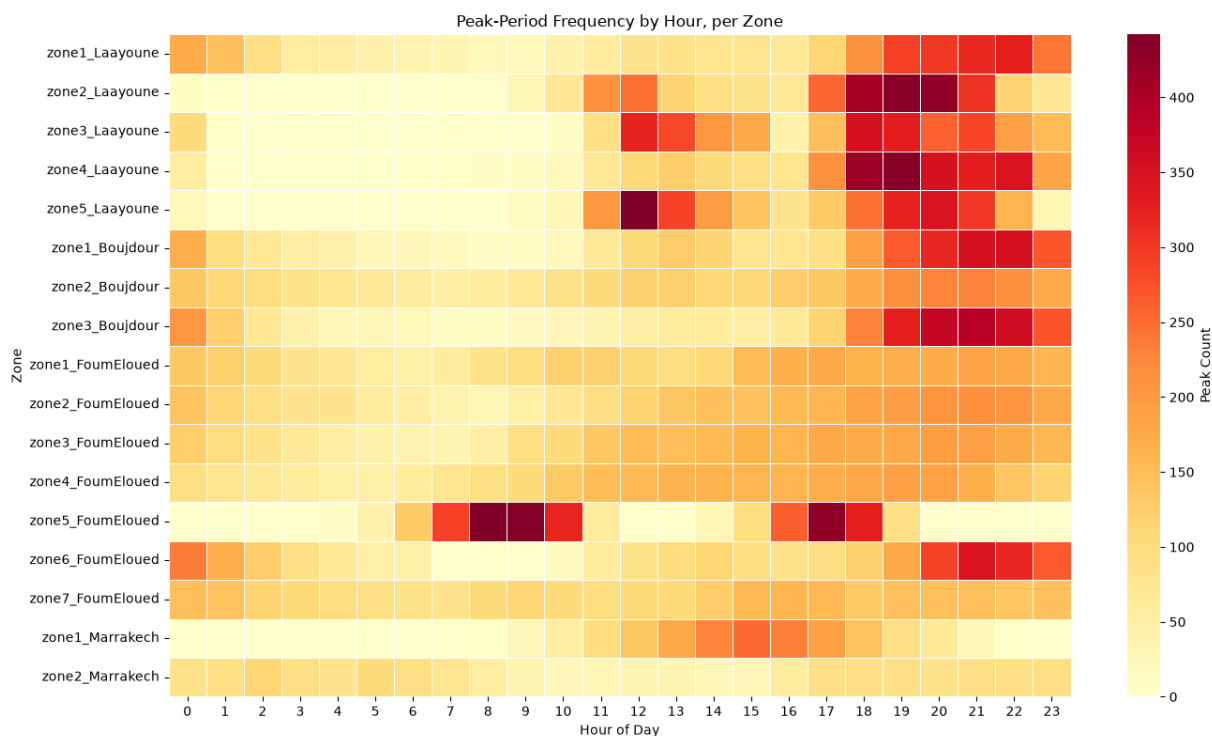
g = sns.catplot(
    data=long_df, x='zone', y='kw', col='city',
    kind='box', sharex=False, sharey=False, col_wrap=2, height=4, aspect=1.3
)
g.set_titles('{col_name}')
g.set_axis_labels('Zone', 'kw')
plt.tight_layout()
plt.show()
```



```
In [10]: # Zone by Hour heatmap
df_kw['hour'] = df_kw.index.hour

heatmap_data = pd.DataFrame(index=range(24), columns=zone_cols, dtype=float)
for col in zone_cols:
    counts = df_kw[df_kw[f'peak_{col}'] == 'Peak'].groupby('hour').size()
    heatmap_data[col] = counts.reindex(range(24), fill_value=0)

plt.figure(figsize=(14, 8))
sns.heatmap(heatmap_data.T, cmap='YlOrRd', linewidths=0.5, cbar_kws={'label': 'Peak'})
plt.title('Peak-Period Frequency by Hour, per Zone')
plt.xlabel('Hour of Day')
plt.ylabel('Zone')
plt.tight_layout()
plt.show()
```

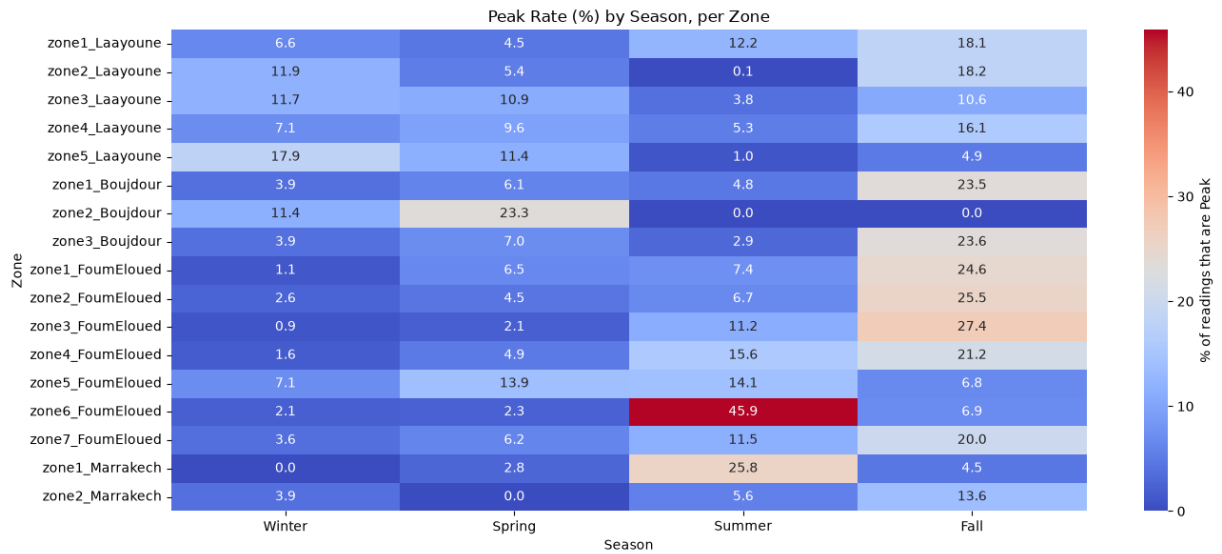


```
In [11]: # seasonal correlation, checking to see if peak usage changes by time of year on wh
def month_to_season(month):
    if month in [12, 1, 2]:
        return 'Winter'
    elif month in [3, 4, 5]:
        return 'Spring'
    elif month in [6, 7, 8]:
        return 'Summer'
    else:
        return 'Fall'

df_kw['season'] = df_kw.index.month.map(month_to_season)

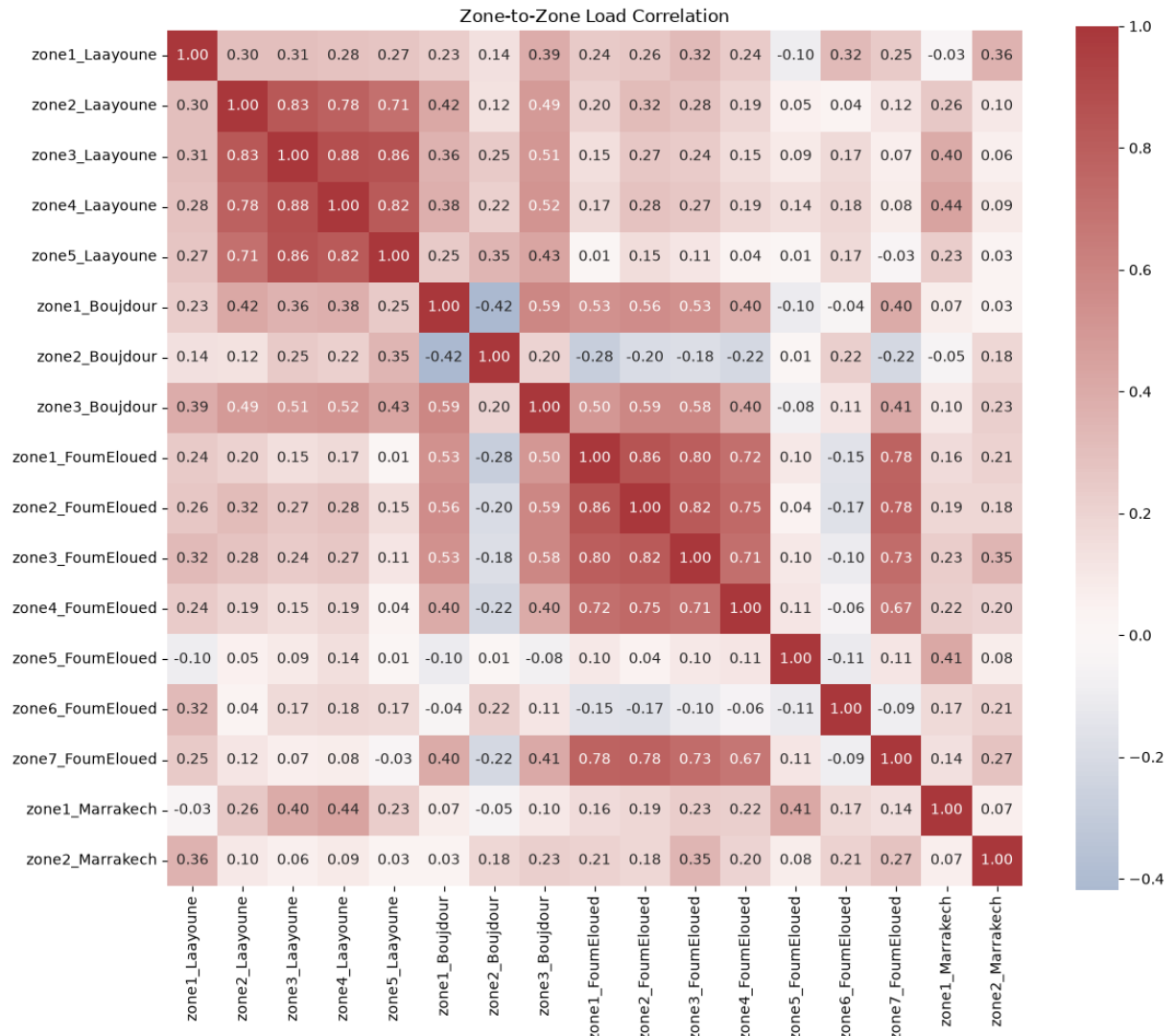
# Peak rate (% of readings that are "Peak") per zone per season
season_peak_rate = pd.DataFrame(index=['Winter', 'Spring', 'Summer', 'Fall'])
for col in zone_cols:
    rate = df_kw.groupby('season')[f'peak_{col}'].apply(lambda s: (s == 'Peak').mean())
    season_peak_rate[col] = rate.reindex(season_peak_rate.index)

plt.figure(figsize=(14, 6))
sns.heatmap(season_peak_rate.T, annot=True, fmt='.1f', cmap='coolwarm', cbar_kws={'
plt.title('Peak Rate (%) by Season, per Zone')
plt.xlabel('Season')
plt.ylabel('Zone')
plt.tight_layout()
plt.show()
```



```
In [12]: # Zone x Zone correlation matrix
corr_matrix = df_kw[zone_cols].corr()

plt.figure(figsize=(12, 10))
sns.heatmap(corr_matrix, annot=True, fmt='.2f', cmap='vlag', center=0, square=True)
plt.title('Zone-to-Zone Load Correlation')
plt.tight_layout()
plt.show()
```



In []: